

Reviewer Pack — Polymatroid Spectral Gap of Monotone DNFs Detects k-Cliques

Entry 1ae8691b7cfd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 19:16:42 UTC

Polymatroid Spectral Gap of Monotone DNFs Detects k-Cliques

Entry ID: 1ae8691b7cfd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 19:16:42 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid expansion via Lasserre level-1 SOS spectral gap of term-variable incidence (Barak-Steurer-style polymatroid rank lift) **Field B** (complexity object): Monotone DNF size lower bounds for the k-CLIQUE indicator (Razborov monotone program)

Statement:

For a monotone DNF $D = T_1 \vee \dots \vee T_s$ on n Boolean variables, let $M_D \in \{0,1\}^{\{s \times n\}}$ be its term-variable incidence matrix ($M[i,v]=1$ iff variable v occurs in term T_i) and let $G_D := D_r^{\wedge -1/2} M_D D_c^{\wedge -1/2}$ be the normalized bipartite incidence (D_r, D_c are the row/column degree diagonals). Define the polymatroid SOS-gap $\mu(D) := \sigma_1(G_D) - \sigma_2(G_D)$, the gap between the top two singular values of the level-1 Lasserre moment lift. Conjecture: (i) μ is submodular under conjunctive merge, $\mu(D \wedge D') \leq \mu(D) + \mu(D')$; (ii) for any monotone DNF D with $s \leq n^c$ terms representing a function with a poly(n)-size monotone circuit, $\mu(D) \leq C \cdot c \cdot \log n$ for an absolute constant C ; (iii) for the canonical CLIQUE $_k$ DNF on K_N (terms = all k -cliques, variables = the $n = C(N,2)$ edges), $\mu \geq c_0 \cdot \sqrt{N}$ for every $k \geq 3$ and some absolute $c_0 > 0$.

Rationale (proposer's reasoning):

Razborov's approximation method replaces $\wedge/\vee$ with lattice operations whose distortion is exactly measured by polymatroid spread over positive/negative test sets; the singular spectrum of the term-variable incidence is the linear-algebraic shadow of this spread at SOS level 1, so a spectral gap predicts an unavoidable polymatroid obstruction to covering k -cliques compactly. The k -clique DNF's terms inherit Johnson-scheme structure $J(N,k)$, whose second eigenvalue is bounded away from the top by an $\sqrt{N}$ -scale gap that random poly-size monotone DNFs cannot reproduce. Because μ acts on the SYNTACTIC representation (not the truth table) it sidesteps natural-proofs largeness, and because it is purely real-spectral on $\{0,1\}$ -matrices it does not algebrize over polynomial extensions of $\text{GF}(2)$.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b70e9d081d57b85b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Over $N \in \{8, 10, 12, 15, 18, 20, 25, 30, 35, 40\}$: (iii) linear fit $\mu_{\text{CLIQUE}}(N)$ vs $\sqrt{N}$ has slope ≥ 0.1 and $R^2 \geq 0.9$, with $\mu_{\text{CLIQUE}}(N) \geq 0.1 \cdot \sqrt{N}$ for every $N \geq 15$; (ii) max over 30 seeds $\times$ all N of $\mu_{\text{RAND}}/\log N \leq 5$; (i) all 30 sampled pairs satisfy $\mu(D \wedge D') \leq \mu(D) + \mu(D')$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.80	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - monotone DNF k-clique lower bound Lasserre SOS spectral gap - Razborov monotone circuit clique polymatroid rank lift incidence-term-variable bipartite incidence singular value monotone formula clique

Top relevant hits considered: - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/2403.19550v2>] Spectral gap for surfaces of infinite volume with negative curvature - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/2501.09545v1>] Hardness of clique approximation for monotone circuits - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/1704.06241v3>] On monotone circuits with local oracles and clique lower bounds - [<http://arxiv.org/abs/2207.14810v3>] Simplifying a classical-quantum algorithm interpolation with quantum singular value transformations - [<http://arxiv.org/abs/2302.03695v4>] The Harer-Zagier and Jackson formulas and new results for one-face bipartite maps

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        # Find pivot
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
```

```

        max_row = j
        A[i], A[max_row] = A[max_row], A[i]

    # Eliminate below the pivot
    for j in range(i + 1, m):
        factor = -A[j][i] / A[i][i]
        for k in range(n):
            if i == k:
                A[j][k] = 0
            else:
                A[j][k] += factor * A[i][k]

# Back substitution
x = [0] * n
for i in range(m - 1, -1, -1):
    x[i] = A[i][-1]
    for j in range(i + 1, n):
        x[i] -= A[i][j] * x[j]
    x[i] /= A[i][i]

return x

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def compute_spectral_gap(A):
    m, n = len(A), len(A[0])
    I = [[Fraction(1 if i == j else 0) for j in range(n)] for i in range(m)]

    # Compute A^T * A
    AT_A = [[sum(A[i][k] * A[k][j] for k in range(n)) for j in range(m)] for i in range(m)]

    # Compute eigenvalues of A^T * A using QR algorithm
    max_iter = 1000
    tol = 1e-6
    Q, R = A, I
    for _ in range(max_iter):
        Q, R = matrix_multiply(Q, R), gaussian_elimination(R)
        A = matrix_multiply(R, Q)
        if all(abs(A[i][j]) < tol for i in range(m) for j in range(i + 1, m)):
            break

    # Extract eigenvalues from diagonal of R
    eigenvalues = [R[i][i] for i in range(min(m, n))]
    return sorted(eigenvalues, reverse=True)

def compute_polymatroid_spectral_gap(n):
    # Construct the canonical CLIQUE_3 DNF
    M = [[0] * (n * (n - 1) // 2) for _ in range(n * (n - 1) // 2)]
    edge_index = 0

```

```

for i in range(n):
    for j in range(i + 1, n):
        M[edge_index][i * (n - 1) // 2 + j - i - 1] = 1
        edge_index += 1

# Compute the normalized bipartite incidence matrix  $G_D$ 
D_r = [sum(row[i] for row in M) for i in range(n * (n - 1) // 2)]
D_c = [sum(M[i][j] for i in range(n * (n - 1) // 2)) for j in range(n)]
D_r_inv = [Fraction(1, d) if d != 0 else 0 for d in D_r]
D_c_inv = [Fraction(1, d) if d != 0 else 0 for d in D_c]

M_normalized = [[M[i][j] * math.sqrt(D_r_inv[i]) * math.sqrt(D_c_inv[j]) for j in range(n)] for i in range(n * (n - 1) // 2)]

# Compute the spectral gap
eigenvalues = compute_spectral_gap(M_normalized)
return eigenvalues[0] - eigenvalues[1]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 15, 18, 20, 25, 30, 35, 40]
    mu_clique_values = []
    mu_rand_values = []

    for n in n_values:
        # Compute  $\mu_{\text{CLIQUE}}(N)$ 
        mu_clique = compute_polymatroid_spectral_gap(n)
        mu_clique_values.append(mu_clique)

        # Compute  $\mu_{\text{RAND}}(N)$ 
        M_rand = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
        eigenvalues = compute_spectral_gap(M_rand)
        mu_rand = eigenvalues[0] - eigenvalues[1]
        mu_rand_values.append(mu_rand)

    # Check submodularity
    submodular = True
    pairs = [(random.choice(n_values), random.choice(n_values)) for _ in range(30)]
    for n1, n2 in pairs:
        M1 = [[random.randint(0, 1) for _ in range(n1)] for _ in range(n1)]
        M2 = [[random.randint(0, 1) for _ in range(n2)] for _ in range(n2)]
        mu_D1 = compute_spectral_gap(M1)
        mu_D2 = compute_spectral_gap(M2)
        mu_D_and_D2 = compute_spectral_gap(matrix_multiply(M1, M2))
        if mu_D_and_D2 > mu_D1 + mu_D2:
            submodular = False
            break

    # Check mean and standard deviation of  $\mu_{\text{CLIQUE}}(N)$  vs  $\sqrt{N}$ 
    mu_clique_mean = sum(mu_clique_values) / len(mu_clique_values)
    mu_clique_std = math.sqrt(sum((x - mu_clique_mean) ** 2 for x in mu_clique_values) / len(mu_clique_values))

    # Check max of  $\mu_{\text{RAND}}/\log N$ 
    mu_rand_max = max(mu_rand_values)

    return {
        "metric_name": "polymatroid_spectral_gap",

```

```

        "metric_value": mu_clique_mean,
        "instances_tested": len(n_values),
        "conjecture_holds": mu_clique_mean >= 0.1 * math.sqrt(15) and mu_rand_max <= 5 and submodu
        "counterexample": "" if mu_clique_mean >= 0.1 * math.sqrt(15) and mu_rand_max <= 5 and sub
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mu_clique_mean = sum(r["metric_value"] for r in results) / len(results)
    mu_clique_std = math.sqrt(sum((r["metric_value"] - mu_clique_mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mu_clique_mean} std={mu_clique_std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"submodularity_violation\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_22556a5e.py", line 149, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_22556a5e.py", line 106, in run_trial
    mu_clique = compute_polymatroid_spectral_gap(n)
                 ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_22556a5e.py", line 95, in compute_polymatroid_spectral_gap
    eigenvalues = compute_spectral_gap(M_normalized)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_22556a5e.py", line 61, in compute_spectral_gap
    AT_A = [[sum(A[i][k] * A[k][j] for k in range(n)) for j in range(m)] for i in range(m)]
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_22556a5e.py", line 61, in <genexpr>
    AT_A = [[sum(A[i][k] * A[k][j] for k in range(n)) for j in range(m)] for i in range(m)]
              ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError due to a matrix dimension bug in `compute_spectral_gap` (indexing `A[i][k]` with `k` in `range(n)` for an incompatible shape), so no data was produced for any of the three claims. | next: Fix the matrix multiplication in `compute_spectral_gap` (use `numpy.linalg.svd` on `G_D` directly) and rerun the pre-registered N sweep and 30-seed protocol.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	195591
2	preregistration	claude_max	opus	0	0	7220
3	novelty	claude_max	opus	0	0	4061
4	novelty	claude_max	opus	0	0	8303
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	99392
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34301
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16448
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21944
9	judge	claude_max	opus	0	0	4745

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 392003 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/1ae8691b7cfd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1ae8691b7cfd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1ae8691b7cfd.tar.gz` (if generated)